

제5강

1) useReducer를 이용한 입출금

- 각 버튼을 클릭할 때 마다 금액만큼 입금, 출금하고 0로 리셋 하고, 이율만큼 금액을 가산하는 프로그램을 작성한다.

[src]-[components05] BankPage.jsx

```
import React, { useState, useReducer, useRef } from 'react'
import './Style05.css'

const ACTION_TYPE = {
  deposit: '입금',
  withdrawal: '출금',
  interest: '이자',
  close: '해지'
}

const initState = 10000; //초기 잔액
const rate = 0.1; //이율

const reducer = (state, action) => {
  const amount = parseInt(action.amount);

  switch(action.type){
    case ACTION_TYPE.deposit:
      return state + amount;
    case ACTION_TYPE.withdrawal:
      if(state < amount){
        alert('잔액이 부족합니다!');
        return state;
      }
      return state - amount;
    case ACTION_TYPE.interest:
      return state + (state * rate);
    case ACTION_TYPE.close:
      return 0;
    default:
      return state;
  }
}

const BankPage = () => {
  const [amount, setAmount] = useState(1000);
  const [state, dispatch] = useReducer(reducer, initState);
  return (
    <div className='box'>
      <h1>잔액:{parseFloat(state).toLocaleString()}원</h1>
      <div>
        <span>금액:</span>
        <input value={amount}
          onChange={(e)=>setAmount(e.target.value)}
          placeholder='금액' type='number' step={1000}/>
      </div>
      <div>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.deposit, amount} )}>입금</button>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.withdrawal, amount} )}>출금</button>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.interest} )}>이자</button>
        <button onClick={()=>dispatch( {type:ACTION_TYPE.close} )}>해지</button>
      </div>
    </div>
  )
}

export default BankPage
```

2) useReducer를 이용한 장바구니

1. 등록 : 장바구니 상품수를 1증가하고 장바구니 목록에 상품을 등록한다.
2. 삭제 : 장바구니 상품수를 1감소하고 장바구니 목록에서 해당 상품을 삭제한다.
3. 주문 : 장바구니 상품수를 1감소하고 장바구니 목록에서 상품을 삭제하고 주문 상품수를 1증가하고 주문 목록에 상품을 등록한다.
4. 취소 : 주문 상품수를 1감소하고 주문 목록에서 상품을 삭제하고 장바구니 상품수를 1증가하고 장바구니 목록에 상품을 등록한다.

[src]-[components05] CartPage.jsx

```
import React, { useReducer, useState, useRef } from 'react'
import Product from './Product'
import './Style05.css'

export const ACTION_TYPE = {
  add:'등록',
  delete:'삭제',
  order:'주문',
  cancel:'취소'
}

const initState = {
  cart: {
    count:1,
    products:[{ id:3, name:'삼성 냉장고'}]
  },
  order: {
    count:2,
    products:[{ id:1, name:'엘지 냉장고'}, {id:2, name:'엘지 세탁기'}]
  }
}

const reducer = (state, action) => {
}

const CartPage = () => {
  const [name, setName] = useState('삼성 세탁기');
  const [state, dispatch] = useReducer(reducer, initState);
  const idRef = useRef(4);

  return (
    <div className='box'>
      <div>
        <h1>카트 목록</h1>
        <h5>상품수:{state.cart.count}개</h5>
        <input value={name} onChange={(e)=>setName(e.target.value)} placeholder='상품명' />
        <button>등록</button>
        <table>
          <tbody>
            { state.cart.products.map(p=>
              <Product key={p.id} product={p} type='cart' dispatch={dispatch}/>
            )}
          </tbody>
        </table>
      </div>
      <div>
        <h1>주문 목록</h1>
        <h5>상품수:{state.order.count}개</h5>
        <table>
          <tbody>
            { state.order.products.map(p=>
              <Product key={p.id} product={p} type='order' dispatch={dispatch}/>
            )}
          </tbody>
        </table>
      </div>
    </div>
  )
}

export default CartPage
```

[src]-[components05] CartPage.jsx

```
const reducer = (state, action) => {
  switch(action.type){
    case ACTION_TYPE.add:
      return {
        ...state,
        cart:{
          count:state.cart.count + 1,
          products:state.cart.products.concat(action.product)
        }
      }
    case ACTION_TYPE.delete:
      return {
        ...state,
        cart:{
          count:state.cart.count - 1,
          products:state.cart.products.filter(p=>p.id !== action.product.id)
        }
      }
    case ACTION_TYPE.order:
      return {
        ...state,
        cart:{
          count:state.cart.count - 1,
          products:state.cart.products.filter(p=>p.id !== action.product.id)
        },
        order:{
          count:state.order.count + 1,
          products:state.order.products.concat(action.product)
        }
      }
    case ACTION_TYPE.cancel:
      return {
        ...state,
        cart:{
          count:state.cart.count + 1,
          products:state.cart.products.concat(action.product)
        },
        order:{
          count:state.order.count - 1,
          products:state.order.products.filter(p=>p.id !== action.product.id)
        }
      }
  }
}

<button onClick={()=>dispatch({type:ACTION_TYPE.add, product:{id:idRef.current++, name}})}>등록</button>
```

[src]-[components05] Product.jsx

```
import { ACTION_TYPE } from './CartPage';
const Product = ({product, type, dispatch}) => {
  const { id, name } = product;
  return (
    <tr>
      <td width='50px'>{id}</td>
      <td width='270px'>{name}</td>
      { type=='cart' &&
        <td>
          <button onClick={()=>dispatch({type:ACTION_TYPE.delete, product})}>삭제</button>
          <button onClick={()=>dispatch({type:ACTION_TYPE.order, product})}>주문</button>
        </td> }
      { type=='order' &&
        <td>
          <button onClick={()=>dispatch({type:ACTION_TYPE.cancel, product})}>취소</button>
        </td> }
    </tr>
  )
}
export default Product
```

제6강

1) Props를 이용한 테마 변경

[src]-[components06]-[props] RootPage.jsx

```
import React, { useState } from 'react'
import '../Style06.css'
import Page from './Page'

const RootPage = () => {
  const [dark, setDark] = useState(false);
  return (
    <div>
      <Page dark={dark} setDark={setDark}/>
    </div>
  )
}
export default RootPage
```

```
.black {
  color:white;
  background: black;
}

.gray {
  color: black;
  background: lightgray;
}

.white {
  color:black;
  background: white;
}
```

[src]-[components06]-[props] Page.jsx

```
const Page = ({dark, setDark}) => {
  return (
    <div>
      <Header dark={dark}/>
      <Content dark={dark}/>
      <Footer dark={dark} setDark={setDark}/>
    </div>
  )
}
export default Page
```

[src]-[components06]-[props] Footer.jsx

```
const Footer = ({dark, setDark}) => {
  return (
    <div className={`footer ${dark ? 'black':'gray'}`}>
      <button onClick={()=>setDark(!dark)}>{dark ? 'Light Mode' : 'Dark Mode'}</button>
    </div>
  )
}
export default Footer
```

[src]-[components06]-[props] Header.jsx

```
const Header = ({dark}) => {
  return (
    <div className={`header ${dark ? 'black':'gray'}`}>
      <h1>Welcom 홍길동</h1>
    </div>
  )
}
export default Header
```

[src]-[components06]-[props] Content.jsx

```
const Content = ({dark}) => {
  return (
    <div className={`content ${dark ? 'black':'white'}`}>
      <h1>홍길동님 좋은 하루되세요!</h1>
    </div>
  )
}
export default Content
```

2) useContext를 이용한 테마, 로그인 상태 변경

[src]-[components06]-[context] RootPage.jsx

```
import React, { useState } from 'react'
import { ThemeContext } from './ThemeContext';
import { UserContext } from './UserContext';
import './Style06.css'
import Page from './Page'

const RootPage = () => {
  const [dark, setDark] = useState(false);
  const [user, setUser] = useState({
    isLogin:false,
    name:'사용자'
  });

  return (
    <UserContext value={{user, setUser}}>
      <ThemeContext.Provider value={{dark, setDark}}>
        <Page/>
      </ThemeContext.Provider>
    </UserContext>
  )
}
export default RootPage
```

[src]-[components06]-[context] Footer.jsx

```
import { useContext } from 'react'
import { ThemeContext } from './ThemeContext'
import { UserContext } from './UserContext'

const Footer = () => {
  const {dark, setDark} = useContext(ThemeContext);
  const {user, setUser} = useContext(UserContext);
  return (
    <div className={`footer ${dark ? 'black':'gray'}`>
      <span>{user.name}</span>
      { user.isLogin ?
        <button onClick={()=>setUser({ isLogin:false, name:'사용자' })}>로그아웃</button>
        :
        <button onClick={()=>setUser({ isLogin:true, name:'홍길동' })}>로그인</button>
      }
      <button onClick={()=>setDark(!dark)}>{dark ? 'Dark Mode':'Light Mode'}</button>
    </div>
  )
}
export default Footer
```

[src]-[components06]-[context] Header.jsx

```
import { useContext } from 'react'
import { ThemeContext } from './ThemeContext'
import { UserContext } from './UserContext';

const Header = () => {
  const {dark} = useContext(ThemeContext);
  const {user} = useContext(UserContext);
  return (
    <div className={`header ${dark ? 'black':'gray'}`>
      <h1>Welcom {user.name}</h1>
    </div>
  )
}
export default Header
```

```
[src]-[components06]-[context] Content.jsx
```

```
import { useContext } from 'react'
import { ThemeContext } from './ThemeContext'
import { UserContext } from './UserContext';

const Content = () => {
  const {dark} = useContext(ThemeContext);
  const {user} = useContext(UserContext);
  return (
    <div className={`content ${dark ? 'black':'white'}`}>
      <h1>{user.name}님 좋은 하루되세요!</h1>
    </div>
  )
}
export default Content
```

3) Provider 컴포넌트 이용

```
[src]-[components06]-[context] CommonProvider.jsx
```

```
import React, { useState } from 'react'
import { UserContext } from './UserContext';
import { ThemeContext } from './ThemeContext';

const CommonProvider = ({children}) => {
  const [dark, setDark] = useState();
  const [user, setUser] = useState({
    isLogin:false,
    name:'사용자'
  });

  return (
    <UserContext.Provider value={{user, setUser}}>
      <ThemeContext.Provider value={{dark, setDark}}>
        {children}
      </ThemeContext.Provider>
    </UserContext.Provider>
  )
}
export default CommonProvider
```

```
[src]-[components06]-[context] RootPage.jsx
```

```
import React, { useState } from 'react'
import { ThemeContext } from './ThemeContext';
import { UserContext } from './UserContext';

import CommonProvider from './CommonProvider';
import '../Style06.css'
import Page from './Page'

const RootPage = () => {
  return (
    <CommonProvider>
      <Page/>
    </CommonProvider>
  )
}
export default RootPage
```

제7강

1) react-router-dom를 이용한 MainRouter

[src]-[components07] MainRouter.jsx

```
import { Link, Routes, Route, NavLink, useLocation } from 'react-router-dom'
import '../Style07.css'
import HomePage from './HomePage'
import AboutPage from './AboutPage'
import ProfilePage from './ProfilePage'
import NaviPage from './NaviPage'

const MainRouter = () => {
  const { pathname } = useLocation();

  return (
    <div className='box'>
      <h5>메뉴를 선택하세요! ({pathname})</h5>
      <ul>
        <li><NavLink to='/'>홈</NavLink></li>
        <li><NavLink to='/about?page=1 &query=노트북&detail=true'>소개</NavLink></li>
        <li><NavLink to='/profiles'>프로필</NavLink></li>
        <li><NavLink to='/info'>Infomation</NavLink></li>
        <li><NavLink to='/navi'>Navigation</NavLink></li>
      </ul>
      <Routes>
        <Route path='/' element={<HomePage/>} />
        <Route path='/about' element={<AboutPage/>} />
        <Route path='/profiles/*' element={<ProfilePage/>} />
        <Route path='/info' element={<AboutPage/>} />
        <Route path='/navi' element={<NaviPage/>} />
      </Routes>
    </div>
  )
}
```

```
export default MainRouter
```

[src]-[components07] ProfilePage.jsx

```
.active {
  background: gray;
  color: white;
}
```

[src]-[components07] ProfilePage.jsx

```
import React from 'react'

const HomePage = () => {
  return (
    <div>
      <h1>홈</h1>
      <div>가장 먼저 보여주는 페이지입니다.</div>
    </div>
  )
}
```

```
export default HomePage
```

[src]-[components07] ProfilePage.jsx

```
const ProfilePage = () => {
  return (
    <div>
      <h5>{username} 프로필</h5>
      <ul>
        <li><NavLink to='/profiles/hong'>홍길동</NavLink></li>
        <li><NavLink to='/profiles/kang'>강참찬</NavLink></li>
        <li><NavLink to='/profiles/sung'>성춘향</NavLink></li>
      </ul>
      <Routes>
        <Route path="/" element={<div>사용자를 선택해 주세요.</div>}/>
        <Route path="/:username" element={<Profile/>}/>
      </Routes>
    </div>
  )
}
```

[src]-[components07] Profile.jsx

```
import { useParams } from 'react-router-dom'
...
const Profile = () => {
  const {username} = useParams();
  return (
    <div>
      <h5>{username} 프로필</h5>
      <div>이름:{data[username].name}</div>
      <div>소개:{data[username].description}</div>
    </div>
  )
}
```

[src]-[components07] AboutPage.jsx

```
const AboutPage = () => {
  const [search] = useSearchParams(); //search 객체의 값들은 모두 문자열 숫자나 boolean인 경우 변환
  const page = parseInt(search.get('page'));
  const query = search.get('query');
  const detail = search.get('detail')==='true';
  return (
    <div>
      <h1>소개</h1>
      <div>리액트 라우터 기초를 실습해 보는 예제 프로젝트입니다.</div>
      {detail && <div>페이지:{page+1}, 검색어:{query}</div> }
    </div>
  )
}
```

[src]-[components07] NaviPage.jsx

```
import React from 'react'
import { useNavigate } from 'react-router-dom'
const NaviPage = () => {
  const navi = useNavigate();
  return (
    <div>
      <h1>Navigation</h1>
      <button onClick={ ()=>navi(-1) }>뒤로</button>
      <button onClick={ ()=>navi('/') }>홈으로</button>
    </div>
  )
}
```

2) react-router-dom를 이용한 JSONRouter

[src]-[components07] JSONRouter.jsx

```
const JSONRouter = () => {  
  return (  
    <div className='box'>  
      <div>  
        <NavLink to='/json/posts'>Posts</NavLink>  
        <NavLink to='/json/todos'>Todos</NavLink>  
        <NavLink to='/json/albums'>Albums</NavLink>  
      </div>  
      <Routes>  
        <Route path='/json/:type' element={<JSONPage/>}/>  
      </Routes>  
    </div>  
  )  
}  
export default JSONRouter
```

[src]-[components07] JSONPage.jsx

```
import React, { useEffect, useState } from 'react'  
import { useNavigate, useParams, useSearchParams } from 'react-router-dom'  
  
const JSONPage = () => {  
  const [response, setResponse] = useState(null);  
  
  const { type } = useParams();  
  const [search] = useSearchParams();  
  const id = parseInt(search.get('id')) || 1;  
  
  const navi = useNavigate();  
  
  const callAPI = () => {  
    fetch(`https://jsonplaceholder.typicode.com/${type}?id=${id}`)  
      .then(response => response.json())  
      .then(json => {  
        console.log(json);  
        setResponse(json);  
      })  
  }  
  
  useEffect(() => {  
    callAPI();  
  }, [type, id]);  
  
  return (  
    <div className='json'>  
      <h5>JSONPlaceholder</h5>  
      <pre>{response && JSON.stringify(response, null, 2)}</pre>  
      <div>  
        <button onClick={()=>navi(`/json/${type}?id=${id-1}`)} disabled={id===1}>감소</button>  
        <span>{id}</span>  
        <button onClick={()=>navi(`/json/${type}?id=${id+1}`)}>증가</button>  
      </div>  
    </div>  
  )  
}  
export default JSONPage
```

제8강

1) 리액트 부트스트랩을 이용한 Layout

[src]-[components08] LayoutPage.jsx

```
import React, { useEffect, useRef, useState } from 'react'
import { Row, Col, Card } from 'react-bootstrap'
import axios from 'axios'
import PageButton from './PageButton';

const LayoutPage = () => {
  const [posts, setPosts] = useState([]);
  const [page, setPage] = useState(1);
  const lastRef = useRef(1);
  const size=12;

  const callAPI = async() => {
    const url = 'https://jsonplaceholder.typicode.com/posts';
    const res = await axios.get(url);
    const start = (page-1) * size + 1;
    const end = page * size;
    const data = res.data.filter(post=>post.id>=start && post.id<=end);
    setPosts(data);
    lastRef.current=Math.ceil(res.data.length/size);
  }

  useEffect(()=>{
    callAPI();
  }, [page]);

  return (
    <div>
      <h1 className='my-5 text-center'>게시글</h1>
      <Row>
        { posts.map(post=>
          <Col key={post.id} xs={6} md={4} lg={3} xl={2} className='mb-3'>
            <Card>
              <Card.Body>
                <div className='body'>{post.body}</div>
              </Card.Body>
              <Card.Footer>
                <div className='title text-truncate'>{post.title}</div>
              </Card.Footer>
            </Card>
          </Col>
        )}
      </Row>
      <PageButton page={page} setPage={setPage} last={lastRef.current}/>
    </div>
  )
}
```

```
.body {
  overflow: hidden;
  text-overflow: ellipsis;
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  font-size: 12px;
}

.title {
  font-size: 12px;
  justify-content: left;
}
```

[src]-[components08] PageButton.jsx

```
import { Button } from 'react-bootstrap'
const PageButton = ({ page, setPage, last }) => {
  return (
    <div className='text-center mt-3'>
      <Button onClick={()=>setPage(page-1)} disabled={page===1}>이전</Button>
      <span> {page}/{last} </span>
      <Button onClick={()=>setPage(page+1)} disabled={page===last}>다음</Button>
    </div>
  )
}
```

2) 리액트 부트스트랩을 이용한 Table

[src]-[components08] TablePage.jsx

```
import { Table } from 'react-bootstrap'
...
const TablePage = () => {
  const [posts, setPosts] = useState([]);
  const [page, setPage] = useState(1);
  const lastRef = useRef(1);
  const size=10;

  const callAPI = async() => {
    const url = 'https://jsonplaceholder.typicode.com/posts';
    const res = await axios.get(url);
    const start = (page-1) * size + 1;
    const end = page * size;
    const data = res.data.filter(post=>post.id>=start && post.id<=end);
    setPosts(data);
    lastRef.current=Math.ceil(res.data.length/size);
  }

  useEffect(()=>{
    callAPI();
  }, [page]);

  return (
    <div>
      <h1 className='text-center my-5'>게시글</h1>
      <Table hover striped bordered>
        <tbody>
          { posts.map(post=>
            <tr key={post.id}>
              <td>{post.id}</td>
              <td>{post.title}</td>
            </tr>
          )}
        </tbody>
      </Table>
      <PageButton page={page} setPage={setPage} last={lastRef.current}/>
    </div>
  )
}
```

[src]-[components08] MainRouter.jsx

```
import ModalProvider from '../context/ModalProvider'
...
const MainRouter = () => {
  return (
    <ModalProvider>
      <Container className='mt-5'>
        <NavLink to='/'>레이아웃</NavLink>
        <NavLink to='/table'>테이블</NavLink>
        <NavLink to='/form'>폼</NavLink>
        <hr/>
        <Routes>
          <Route path='/' element={<LayoutPage/>}/>
          <Route path='/table' element={<TablePage/>}/>
          <Route path='/form' element={<FormPage/>}/>
        </Routes>
      </Container>
    </ModalProvider>
  )
}
```

3) 리액트 부트스트랩을 이용한 Form

[src]-[components08] FormPage.jsx

```
import { ModalContext } from '../context/ModalContext'
import { Row, Col, Form, InputGroup, Button } from 'react-bootstrap'
...
const FormPage = () => {
  const formRef = useRef(initForm);
  const { showAlert, setConfirm } = useContext(ModalContext);
  ...
  const onSubmit = (e) => {
    e.preventDefault();
    if(id===''){
      showAlert({show:true, message:'아이디를 입력하세요!'});
    }else{
      setConfirm({
        show:'true',
        message:'정말로 가입하실래요?',
        action:()=>{
          console.log(form);
        }
      });
    }
  }

  const onReset = (e) => {
    e.preventDefault();
    setConfirm({
      show:'true',
      message:'정말로 취소하실래요?',
      action:()=>{
        setForm(formRef.current);
        console.log(form);
      }
    });
  }

  const onChange = (e) => {
    setForm({...form, [e.target.name]:e.target.value});
  }

  return (
    <div>
      <h1 className='text-center my-5'>회원가입</h1>
      <Row className='justify-content-center'>
        <Col xs={12} md={10} lg={9} xl={8}>
          <Form onSubmit={onSubmit} onReset={onReset}>
            <InputGroup className='mb-2'>
              <InputGroup.Text>아이디</InputGroup.Text>
              <Form.Control value={id} onChange={onChange} name='id' placeholder='아이디' />
            </InputGroup>
            ...
            <Form.Control value={intro} onChange={onChange}
              name='intro' placeholder='자기소개' as='textarea' rows={5} />
          <div className='text-center mt-3'>
            <Button disabled={JSON.stringify(form)===JSON.stringify(formRef.current)}
              type='submit' className='px-3 me-2'>가입</Button>
            <Button disabled={JSON.stringify(form)===JSON.stringify(formRef.current)}
              type='reset' className='px-3' variant='secondary'>취소</Button>
          </div>
        </Form>
      </Col>
    </Row>
  </div>
  )
}
```

제9강

1) 카카오 API를 활용한 도서검색

TodoList09

1. HTTP 비동기 통신을 쉽게 해주는 `axios` 자바스크립트 라이브러리를 설치한다.
`npm install axios`
2. 카카오 개발자센터 애플리케이션을 생성한다.
<https://developers.kakao.com/>
3. 내 애플리케이션의 REST API 키를 복사한다.
4. 루트 `.env` 파일에 복사한 키를 붙여 넣은 후 서버를 다시 구동한다.
`REACT_APP_KAKAOAK=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
5. `.gitignore` 파일에 `.env`를 추가하여 github에 업로드 되지 않도록 한다.
6. `BookPage` 컴포넌트에서 검색기능과 페이지 기능을 작성한다.

[src]-[components09] BookPage.jsx

```
const BookPage = () => {
  const [loading, setLoading] = useState(false);
  const [response, setResponse] = useState(null);
  const size=12;
  const query='리액트';
  const page=1;

  const callAPI = async() => {
    setLoading(true);
    const url = 'https://dapi.kakao.com/v3/search/book?target=title';
    const config = {
      headers: { 'Authorization': `KakaoAK ${process.env.REACT_APP_KAKAOAK}` },
      params: { size, query, page }
    };

    const res = await axios(url, config);
    setResponse(res.data);
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, []);

  if(loading || !response) return <h1 className='text-center my-5'>로딩중....</h1>
  const { documents, meta } = response;

  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>도서검색</h1>
      <Row>
        {documents.map(book=>
          <Col key={book.isbn} xs={6} md={4} lg={3} xl={2} className='mb-3'>
            <Card>
              <Card.Body>
                <img src={book.thumbnail} width='100%' />
              </Card.Body>
              <Card.Footer>
                <div className='text-truncate'>{book.title}</div>
              </Card.Footer>
            </Card>
          </Col>
        )}
      </Row>
    </div>
  );
}

export default BookPage;
```

2) 페이징 기능과 검색기능 추가

[src]-[components09] BookPage.jsx

```
const BookPage = () => {
  const [loading, setLoading] = useState(false);
  const [response, setResponse] = useState(null);
  const [text, setText] = useState('');

  const [search] = useSearchParams();
  const size = 12;
  const query = search.get('query') || '리액트';
  const page = parseInt(search.get('page')) || 1;
  const navi = useNavigate();

  const callAPI = async() => {
    setLoading(true);
    const url = 'https://dapi.kakao.com/v3/search/book?target=title';
    const config = {
      headers: {'Authorization': `KakaoAK ${process.env.REACT_APP_KAKAOAK}`},
      params: {size, query, page}
    }
    const res = await axios(url, config);
    console.log(res.data);
    setResponse(res.data);
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, [page, query]);

  const onSubmit = (e) => {
    e.preventDefault();
    navi(`/book?query=${text}`);
  }

  if(loading || !response) return <h1 className='text-center my-5'>로딩중....</h1>
  const { documents, meta } = response;
  const last = Math.ceil(meta.pageable_count/12);

  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>도서검색</h1>
      <Row className='mb-3'>
        <Col xs={6} md={5} lg={4} xl={3}>
          <Form onSubmit={onSubmit}>
            <InputGroup>
              <Form.Control value={text} onChange={(e)=>setText(e.target.value)} />
              <Button type='submit'>검색</Button>
            </InputGroup>
          </Form>
        </Col>
      </Row>
      <Row>
        <div>
          {documents.map(book=>
            ...
          )}
        </div>
        <div className='text-center my-3'>
          <Button disabled={page===1} onClick={()=>navi(`/book?page=${page-1}&query=${query}`)}>이전</Button>
          <span className='mx-3'>{page}/{last}</span>
          <Button disabled={page===last} onClick={()=>navi(`/book?page=${page+1}&query=${query}`)}>다음</Button>
        </div>
      </div>
    </div>
  );
}
```

3) 페이징 기능과 검색기능 분리

[src]-[components09]-[common] SearchForm.jsx

```
const SearchForm = ({ query }) => {
  const { pathname } = useLocation();
  const navi=useNavigate();
  const textRef = useRef(null);
  const [text, setText] = useState(query);

  const onSubmit = (e) => {
    e.preventDefault();
    navi(`${pathname}?query=${text} &page=1`);
  }
  useEffect(() => {
    textRef.current.focus();
  }, []);

  return (
    <Row className='mb-2'>
      <Col xs={6} md={5} lg={4} xl={3}>
        <Form onSubmit={onSubmit}>
          <InputGroup>
            <Form.Control ref={textRef} value={text} onChange={(e) => setText(e.target.value)} placeholder='검색어' />
            <Button type='submit'>검색</Button>
          </InputGroup>
        </Form>
      </Col>
    </Row>
  )
}
```

[src]-[components09]-[common] PagingButton.jsx

```
const PagingButton = ({ page, last, query }) => {
  const navi = useNavigate();
  const { pathname } = useLocation();

  return (
    <div className='text-center mt-3'>
      <Button onClick={() => navi(`${pathname}?page=${page-1} &query=${query}`)} disabled={page===1}>이전</Button>
      <span className='mx-3'>{page} / {last}</span>
      <Button onClick={() => navi(`${pathname}?page=${page+1} &query=${query}`)} disabled={page===last}>다음</Button>
    </div>
  )
}
```

[src]-[components09] BookPage.jsx

```
import SearchForm from '../common/SearchForm';
import PagingButton from '../common/PagingButton';

const BookPage = () => {
  return (
    <div className='my-5'>
      <SearchForm query={query} />
      <Row>
        <Col key={book.isbn} xs={6} md={4} lg={3} xl={2} className='mb-3'>
          ...
        </Col>
      </Row>
      { last > 1 && <PagingButton page={page} last={last} query={query} /> }
      { documents.length === 0 && <h3 className='text-center my-5'>검색 도서가 없습니다.</h3> }
    </div>
  )
}
```

제10강

1) 네이버 API를 활용한 상품검색

ToDoList10

1. BookPage 도서검색에서 BookModal 도서정보 모달창을 만들어 연결한다.
2. 네이버 개발자 센터에서 애플리케이션을 생성한다.
<https://developers.naver.com/main/>
3. 내 애플리케이션에서 NAVERID, NAVERSECRET 키를 복사한다.
4. 루트 .env 파일에 복사한 키를 붙여넣는다.
`REACT_APP_NAVERID=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
`REACT_APP_NAVERSECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
* 교차 출처 리소스 공유(CORS:Cross-Origin Resource Sharing)란?
 - 한 출처(Origin)에서 다른 출처의 리소스에 접근할 수 있도록 허용.
 - 네이버 API 서버는 CORS을 허용하지 않아서 에러발생.
 - 클라이언트와 서버 사이에서 중개자 역할을 하는 프락시 서버 설정 필요.
5. 네이버 검색을 위해 프락시 서버를 package.json에 추가한 후 서버를 다시 구동한다.
`"proxy": "https://openapi.naver.com/v1"`
6. ShopPage 컴포넌트에 검색, 페이지 기능을 추가한다.
7. ShopPage에서 ShopModal 상품정보 컴포넌트를 연결한다.

[src]-[components10] MainRouter.jsx

```
{
  "name": "react-project",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    ...
  },
},
{
  "proxy": "https://openapi.naver.com/v1"
}
```

[src]-[components10] MainRouter.jsx

```
import { Route, Routes } from 'react-router'
import MenuBar from './MenuBar'
import HomePage from './HomePage'
import BookPage from './BookPage'
import ShopPage from './ShopPage'
import ModalProvider from '../context/ModalProvider'
import '../Style.css'
import { Container } from 'react-bootstrap'

const MainRouter = () => {
  return (
    <ModalProvider>
      <Container>
        <MenuBar/>
        <Routes>
          <Route path="/" element={<HomePage/>}/>
          <Route path="/book" element={<BookPage/>}/>
          <Route path="/shop" element={<ShopPage/>}/>
        </Routes>
      </Container>
    </ModalProvider>
  )
}
export default MainRouter
```

[src]-[components10] ShopPage.jsx

```
import React, { useEffect, useState } from 'react'
import axios from 'axios';
import { Row, Col, Card } from 'react-bootstrap';
import { useSearchParams } from 'react-router-dom';
import SearchForm from '../common/SearchForm'
import PagingButton from '../common/PagingButton'

const ShopPage = () => {
  const [loading, setLoading] = useState(false);
  const [response, setResponse] = useState(null);

  const [search] = useSearchParams();
  const size=12;
  const query = search.get('query') || '노트북';
  const page = parseInt(search.get('page') || 1);

  const callAPI = async() => {
    setLoading(true);
    const url = '/search/shop.json';
    const config = {
      headers:{
        'X-Naver-Client-Id':process.env.REACT_APP_NAVERID,
        'X-Naver-Client-Secret':process.env.REACT_APP_NAVERSECRET
      },
      params:{ start:1, query, display:size }
    }
    const res = await axios(url, config);
    console.log(res.data);
    setResponse(res.data);
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, [query, page]);

  if(loading || !response) return <h1 className='text-center my-5'>로딩중...</h1>
  const { items, total } = response;
  const last = Math.ceil(total/size);

  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>상품검색</h1>
      <SearchForm query={query}/>
      <Row>
        {items.map(product=>
          <Col key={product.id} xs={6} md={4} lg={3} xl={2} className='mb-3'>
            <Card>
              <Card.Body>
                <img src={ product.image } width='100%' />
              </Card.Body>
              <Card.Footer>
                <div className='text-truncate title' dangerouslySetInnerHTML={{__html:shop.title}} />
              </Card.Footer>
            </Card>
          </Col>
        )}
      </Row>
      { items.length===0 && <h3 className='text-center my-5'>검색 결과가 없습니다.</h3>}
      {last > 1 && <PagingButton page={page} last={last} query={query}/>}
    </div>
  )
}

export default ShopPage
```

2) 도서정보 모달창

[src]-[componetns10]-[book] BookModal.jsx

```
const BookModal = ({ book }) => {
  const [show, setShow] = useState(false);
  const handleClose = () => setShow(false);
  const handleShow = () => setShow(true);

  return (
    <
      <img style={{cursor:'pointer'}} onClick={handleShow}
        src={book.thumbnail || 'http://placeholder.co/100x150'} width='100%'/>
      <Modal size='lg' show={show} onHide={handleClose} backdrop="static" keyboard={false}>
        <Modal.Header closeButton>
          <Modal.Title>도서정보</Modal.Title>
        </Modal.Header>
        <Modal.Body>
          <Row>
            <Col lg={4} lassName='align-self-center'>
              <img src={book.thumbnail || 'http://placeholder.co/100x150'} width='100%'/>
            </Col>
            <Col className='align-self-center'>
              <h5>{book.title}</h5>
              <hr/>
              <div className='mb-3'>저자: {book.authors}</div>
              <div className='mb-3'>가격: {parseInt(book.price).toLocaleString()}원</div>
              <div className='mb-3'>출판사: {book.publisher}</div>
              <div className='mb-3'>ISBN: {book.isbn}</div>
              <div className='mb-3'>날짜: {book.datetime.substr(0, 10)}</div>
              <div className='mb-3'>상태: {book.status}</div>
            </Col>
          </Row>
          <hr/>
          <div style={{whiteSpace:'pre-wrap'}}>{book.contents || '내용 없음'}</div>
        </Modal.Body>
        <Modal.Footer>
          <Button variant="secondary" onClick={handleClose}>Close</Button>
        </Modal.Footer>
      </Modal>
    </>
  )
}
```

export default BookModal

[src]-[componetns10]-[book] BookModal.jsx

```
import BookModal from './book/BookModal'
...
const BookPage = () => {
  return (
    ...
    <Card>
      <Card.Body>
        <BookModal book={book}/>
      </Card.Body>
      <Card.Footer>
        <div className='title text-truncate'>
          { (page-1)*size + index+1 }. { book.title }
        </div>
      </Card.Footer>
    </Card>
    ...
  )
}
```

export default BookModal

3) 상품정보 모달창

[src]-[componetns10]-[shop] ShopModal.jsx

```
const ShopModal = ({ shop }) => {
  const [show, setShow] = useState(false);
  const handleClose = () => setShow(false);
  const handleShow = () => setShow(true);
  return (
    <>
      <img style={{cursor:'pointer'}} onClick={handleShow} src={shop.image} width='100%' />
      <Modal show={show} onHide={handleClose} backdrop="static" keyboard={false} size='lg'>
        <Modal.Header closeButton>
          <Modal.Title>상품정보</Modal.Title>
        </Modal.Header>
        <Modal.Body>
          <Row>
            <Col lg={4} className='align-self-center'>
              <img src={shop.image || 'http://placeholder.co/100x150'} width='100%' />
            </Col>
            <Col className='align-self-center'>
              <div dangerouslySetInnerHTML={{__html:shop.title}}/>
              <hr/>
              <div className='mb-2'>가격: {shop.lprice ? parseInt(shop.lprice).toLocaleString(): '미정'}</div>
              <div className='mb-2'>상품ID: {shop.productId}</div>
              <div className='mb-2'>제조사: {shop.maker}</div>
              <div className='mb-2'>판매점: {shop.mallName}</div>
              <div className='mb-2'>카테고리: {shop.category1}</div>
            </Col>
          </Row>
        </Modal.Body>
        <Modal.Footer>
          <Button variant="secondary" onClick={handleClose}>Close</Button>
        </Modal.Footer>
      </Modal>
    </>
  );
}
```

export default ShopModal

[src]-[componetns10] BookPage.jsx

```
import ShopModal from './shop/ShopModal'
...
const ShopPage = () => {
  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>상품검색</h1>
      <Row>
        { items.map(shop=>
          <Col key={shop.productId} xs={6} md={4} lg={3} xl={2} className='mb-3'>
            <Card>
              <Card.Body>
                <ShopModal shop={shop}/>
              </Card.Body>
              <Card.Footer>
                <div className='text-truncate title' dangerouslySetInnerHTML={{__html:shop.title}}/>
              </Card.Footer>
            </Card>
          </Col>
        )}
      </Row>
    </div>
  )
}
```